

Lab 8 Solution

Class

```
class opcode_processor_class;

    rand logic [3:0] opcode;
    rand logic [7:0] data;
    bit clk;

    // Holds opcode value post-randomize for transition constraint
    bit [3:0] opcode_post_rand;

    function void post_randomize();
        opcode_post_rand = opcode;
    endfunction

    // Steers opcode to STORE whenever previous opcode was LOAD
    constraint load_store_seq_c {
        (opcode_post_rand == 4'd1) -> (opcode == 4'd2);
    }
    covergroup cg;
        option.per_instance = 1;
        // 1. Auto bins - capped at 5
        cp_opcode : coverpoint opcode {
            option.auto_bin_max = 5;
        }
        // 2. User-defined bins grouping opcodes by function
        cp_opcode_bins : coverpoint opcode {
            bins alu_ops = {4, 5, 6};
            bins load_store = {1, 2};
            bins misc_ops = {0, 3, 7, 8};
            ignore_bins reserved = {9, 10, 11, 12};
            illegal_bins error = {15};
        }
        // 3. Data range bins
        cp_data_range : coverpoint data {
            bins low = {[0 : 63]};
            bins mid = {[64 : 127]};
            bins high = {[128 : 255]};
        }
        // 4. Transition bins - weight 0 excludes them from coverage percentage
        cp_transitions : coverpoint opcode {
            option.weight = 0;
            bins load_to_store = (1 => 2);
            bins store_to_alu = (2 => 4 => 5);
        }
        // 5. Conditional coverpoints - sampled only when iff guard is true
        cp_data_when_alu : coverpoint data iff (opcode inside {4, 5, 6}) {
            bins low = {[0 : 63]};
            bins mid = {[64 : 127]};
            bins high = {[128 : 255]};
        }
        cp_data_when_load_store : coverpoint data iff (opcode inside {1, 2}) {
            bins low = {[0 : 63]};
            bins mid = {[64 : 127]};
            bins high = {[128 : 255]};
        }
    endgroup : cg

    function new();
        cg = new();
    endfunction

endclass : opcode_processor_class
```

Testbench:

```
module top;

    bit        clk;
    logic       rst_n;
    logic [3:0] opcode;
    logic [7:0] data;
    logic [7:0] result;

    opcode_processor_class cov_test_obj;
    int unsigned iter_count;

    opcode_processor dut (.*);

    initial begin
        clk = 1'b0;
        forever #2ns clk = ~clk;
    end

    initial begin
        cov_test_obj = new();

        rst_n = 1'b0;
        #4ns;
        rst_n = 1'b1;
        #2ns;

        iter_count = 0;
        do begin
            iter_count++;
            @(negedge clk);
            assert(cov_test_obj.randomize()) else $finish;
            opcode = cov_test_obj.opcode;
            data = cov_test_obj.data;
            cov_test_obj.cg.sample();
        end while (cov_test_obj.cg.get_coverage() < 100.0);

        $display("\n////////////////////////////////////////");
        $display("No. of iterations to reach 100 coverage = %0d", iter_count);
        $display("////////////////////////////////////////");
        $stop;
    end

endmodule
```

Do File:

```
vlib work
vlog *.v +cover=f -covercells
vsim -voptargs=+acc work.top -cover
add wave *
coverage save top.ucdb -onexit
run -all
#quit -sim
#vcover report top.ucdb -details -annotate -all -output top.txt
```

Log:

```
# Loading work.opcode_processor(fast)
# ** Error: (vsim-8565) Illegal state bin was hit at value=15. The bin counter for the illegal bin '\top_sv_unit::opcode_processor_class::cq.cp_opcode_bins.error' is 1.
# Time: 32 ns Iteration: 1 Instance: /top
# ** Error: (vsim-8565) Illegal state bin was hit at value=15. The bin counter for the illegal bin '\top_sv_unit::opcode_processor_class::cq.cp_opcode_bins.error' is 2.
# Time: 48 ns Iteration: 1 Instance: /top
# ** Error: (vsim-8565) Illegal state bin was hit at value=15. The bin counter for the illegal bin '\top_sv_unit::opcode_processor_class::cq.cp_opcode_bins.error' is 3.
# Time: 104 ns Iteration: 1 Instance: /top
# ** Error: (vsim-8565) Illegal state bin was hit at value=15. The bin counter for the illegal bin '\top_sv_unit::opcode_processor_class::cq.cp_opcode_bins.error' is 4.
# Time: 136 ns Iteration: 1 Instance: /top
#
# ////////////////////////////////////////////////////////////////////
# No. of iterations to reach 100 coverage = 36
# ////////////////////////////////////////////////////////////////////
# Note: $stop : top.sv(42)
# Time: 148 ns Iteration: 1 Instance: /top
```

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_Instances	Get_inst_coverage	Comment
/top_sv_unit/opcode_processor_class		100.00%							
TYPE cg		100.00%	100	100.00%					
CVP cp::cp_opcode		100.00%	100	100.00%					auto(0)
CVP cp::cp_opcode_bins		100.00%	100	100.00%					
CVP cp::cp_data_range		100.00%	100	100.00%					
CVP cp::cp_transitions		0.00%	100	0.00%					
CVP cp::cp_data_when_alu		100.00%	100	100.00%					
CVP cp::cp_data_when_load_store		100.00%	100	100.00%					
INST \top_sv_unit::opcode_processor_class::cq		100.00%	100	100.00%					0
CVP cp_opcode		100.00%	100	100.00%					
bin auto[0:2]		8	1	100.00%					
bin auto[3:5]		5	1	100.00%					
bin auto[6:8]		3	1	100.00%					
bin auto[9:11]		6	1	100.00%					
bin auto[12:15]		14	1	100.00%					
CVP cp_opcode_bins		100.00%	100	100.00%					
illegal_bin error		4	-	-					
ignore_bin reserved		10	-	-					
bin alu_ops		5	1	100.00%					
bin load_store		3	1	100.00%					
bin misc_ops		8	1	100.00%					
CVP cp_data_range		100.00%	100	100.00%					
bin low		5	1	100.00%					
bin mid		10	1	100.00%					
bin high		21	1	100.00%					
CVP cp_transitions		0.00%	100	0.00%					
bin load_to_store		0	1	0.00%					
bin store_to_alu		0	1	0.00%					
CVP cp_data_when_alu		100.00%	100	100.00%					
bin low		2	1	100.00%					
bin mid		1	1	100.00%					
bin high		2	1	100.00%					
CVP cp_data_when_load_store		100.00%	100	100.00%					
bin low		1	1	100.00%					
bin mid		1	1	100.00%					
bin high		1	1	100.00%					